



SAPIENZA
UNIVERSITÀ DI ROMA

Emulatore CUDA

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Laurea Triennale in Ingegneria Informatica

Gabriele Guido

Matricola 2144286

Relatore

Prof. Giorgio Grisetti

Anno Accademico 2023/2024

Emulatore CUDA

Tesi di Laurea. Sapienza Università di Roma

© 2026 Gabriele Guido. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: guido.2144286@studenti.uniroma1.it

Dedicato a nonna

Sommario

La programmazione in CUDA (*Compute Unified Device Architecture*) è il modo più efficace per scrivere programmi eseguibili su GPU (*Graphics Processing Unit*). L'architettura delle schede grafiche consente numerose operazioni parallele, riducendo i tempi di esecuzione e aumentando il **throughput** (attività completate nell'unità di tempo). Negli ultimi anni l'utilizzo di questa tecnologia si è diffuso ulteriormente grazie alla velocità di esecuzione di calcoli sulle matrici e programmare in questa modalità è diventata quindi una competenza richiesta. Il suo punto di forza tuttavia è ciò che rende la progettazione degli algoritmi meno semplice: i dati sono elaborati in parallelo e senza ordine perciò durante l'esecuzione potrebbero verificarsi delle dipendenze dati che, se non trattate correttamente, porterebbero a risultati sbagliati. In CUDA abbandoniamo il concetto di esecuzione sequenziale: in ogni istante un numero finito di thread esegue lo stesso codice su una porzione dell'input dato al programma. Senza un ordine tra questi diventa necessario disegnare algoritmi in cui i thread lavorino in modo indipendente tra loro.

Il progetto nasce dal bisogno di programmare in CUDA anche senza una scheda grafica sul proprio PC e consiste in una libreria in cui ci sono le funzionalità essenziali per testare la correttezza di un algoritmo destinato al calcolo parallelo su GPU. A livello prestazionale non c'è alcun vantaggio, la vera utilità sta nella scrittura e verifica del funzionamento di algoritmi destinati al calcolo parallelo.

Indice

1	CUDA	1
1.1	In poche parole	1
1.2	Griglia, Blocchi e Thread	2
2	Algoritmi	3
2.1	ReductionSum	3
2.2	PrefixSum	3
3	MyCuda	5
3.1	Parametri	5
3.2	Strumenti Principali	5
3.2.1	MyCudaItem	5
3.2.2	MyCudaKernelLaunch()	6
3.3	Algoritmi	7
3.3.1	MyCudaReduction	7
3.3.2	MyCudaPrefixSum	9
3.4	Funzioni Ausiliare	12
3.4.1	MyCudaMalloc()	12
3.4.2	MyCudaMemSwap()	13
3.4.3	MyCudaMemFree()	13
3.4.4	MyCudaBuffersPrint()	14
3.4.5	MyCudaIterations()	14
3.4.6	MyCudaSetThreadArgs()	14
3.4.7	MyCudaIsPowerTwo()	15
3.4.8	MyCudaPrefixSumTest()	15
4	Uso e sviluppi	17

CUDA

1.1 In poche parole

Per capire come funziona l'esecuzione di un programma su una GPU dobbiamo abbandonare la concezione sequenziale e iniziare a pensare a cosa succede nello spazio più che nel tempo. Prenderemo come esempio di input da processare dei buffer di N numeri (interi). Supponiamo di voler eseguire una moltiplicazione per uno scalare S in ogni cella di $src[N]$ e salvare il risultato su $dest[N]$. Immaginiamo di dover progettare un tradizionale algoritmo che verrà lanciato su CPU: non serve altro che iterare l'operazione su tutte le N posizioni del nostro vettore. Supponiamo di avere un processore a singolo **core** e che il nostro sia l'unico processo in CPU. Le istruzioni verranno eseguite in modo sequenziale, una dopo l'altra. La complessità del nostro algoritmo è lineare nella dimensione N del vettore in input ($O(N)$).

```

1   for(int i = 0; i < N; i++)
2       dest[i] = S * src[i];

```

Volendo ottenere lo stesso risultato su una GPU, iterare in questo modo non avrebbe senso: per sfruttare la potenza di calcolo dobbiamo ottimizzare l'esecuzione. In questo esempio dobbiamo immaginare di avere a disposizione N core contemporaneamente e di poter assegnare ogni singola istruzione del ciclo a ognuno di questi, ed è effettivamente quello che accade all'interno di una GPU: ogni **thread** (corrispondente a un singolo core) esegue lo stesso codice su una singola posizione dell'array.

```

1   dest[idx] = S * src[idx];

```

I thread lavorano veramente in parallelo e in questo modo il costo di questo semplice programma passa da essere lineare ($O(n)$) a costante ($O(1)$).

1.2 Griglia, Blocchi e Thread

Per poter gestire input con dimensione maggiore del numero di core (e quindi thread) che abbiamo a disposizione, l'esecuzione viene spezzata in blocchi: ogni blocco è grande quanto il numero massimo di thread. L'input viene processato come una griglia di blocchi quindi CUDA mette a disposizione dei programmatori le informazioni come numero di thread (in un blocco), numero di blocco (nella griglia) e dimensione del blocco. In questo modo possiamo tenere conto dello spostamento spaziale della nostra esecuzione, processando il nostro input come se fosse composto da tante mattonelle (da non confondere con le celle di un array) passate una alla volta al processore.

Algoritmi

Nella libreria sono presenti già due algoritmi usati per testare la correttezza delle funzioni. Vediamo in generale cosa sono e a che servono. L'implementazione nella libreria si trova al capitolo di MyCuda, qui mostrerò una realizzazione sequenziale equivalente (sempre in C) per capirne il meccanismo.

Nota: prenderò come input di esempio degli array di interi per coerenza con la libreria.

2.1 ReductionSum

Prende in input un vettore (numerico) e restituisce la somma di tutte le componenti.

```
1  int SequentialReductionSum(int* src, int size){
2      int sum = 0;
3      for(int i = 0; i<size,i++){
4          sum += src[i];
5      }
6      return sum;
7  }
```

2.2 PrefixSum

Prende in input un vettore (numerico) e scrive in ogni cella *i-esima* la somma dei valori nelle posizioni da 0 a i del vettore.

```
1  void SequentialPrefixSum(int* src, int size){
2      int sum = 0;
3      for(int i = 0; i<size; i++){
4          sum += src[i];
5          scr[i] = sum;
6      }
7  }
```


MyCuda

La libreria MyCuda contiene delle funzionalità utili alla progettazione e simulazione del calcolo parallelo. L'esecuzione effettiva è gestita tramite un numero limitato di thread che simulano i core di una GPU. Il funzionamento generale è gestito tramite una struttura dati *MyCudaItem* al cui interno troviamo i buffer di memoria (con rispettiva dimensione) e il puntatore alla funzione scritta in "*pseudo-CUDA*" che verrà eseguita simulando i core della GPU. Nella libreria ci sono anche diverse funzioni ausiliare che facilitano la fase di sviluppo e verifica: gestione dei 2 buffer (creazione, stampa ed eliminazione), il calcolo di valori speciali (dimensione griglia) e verifica di condizioni necessarie (dimensione buffer). Il cuore della libreria consiste nella funzione che simula la chiamata al *Kernel* della GPU e tutta la gestione, tramite strutture dati dedicate, di indici, parametri e argomenti usati dai thread. Al di là quindi delle varie funzionalità operative, l'essenza di questa libreria è la possibilità di scrivere funzioni (in C) similmente a come si farebbe usando CUDA.

Nota: L'intera libreria è stata pensata per trattare array di interi ma la meccanica può essere facilmente estesa ad altri tipi di dato.

3.1 Parametri

Nell'header *myCuda.h* troviamo le **macro** usate nella libreria.

- **BLOCK_DIM:** dimensione di ogni blocco e quindi il numero massimo di thread generati;
- **DEBUG:** quando è maggiore di 0 attiva le stampe intermedie nelle funzioni, rendendo più facile l'ispezione;
- **SLEEPING_TIME:** nei test ho rallentato le stampe con delle `sleep()` per rendere più leggibile il terminale;
- **RANDOM_RANGE:** indica fino a che numero possono arrivare i numeri pseudocasuali generati come input per i test.

3.2 Strumenti Principali

3.2.1 MyCudaItem

In CUDA il "*workflow*" tipico prevede l'allocazione su GPU, tramite chiamate a sistema, di due zone di memoria grandi uguali: una in cui viene copiato l'input

e l'altra in cui si troverà il risultato. Una volta allocati, il kernel della GPU può procedere all'esecuzione e quando ha finito la CPU ricopia il risultato nella memoria della nostra macchina principale.

Nella mia simulazione non avrebbe senso aggiungere un meccanismo simile: la memoria su cui lavoriamo è sempre la stessa quindi basta allocare due buffer che useremo in tutte le funzioni. Per semplificare il tutto ho scelto di usare una struttura dati in cui salvare gli elementi più ricorrenti, rendendo tutte le funzioni consistenti con questo tipo di dato. Le funzioni di libreria avranno sempre, come parametro, un puntatore a un oggetto di questo tipo, lanciando un errore in caso non sia rispettata questa sintassi.

```
1 struct MyCudaItem{
2     int buffer_size;
3     int size;
4     int* src_buffer;
5     int* dest_buffer;
6     KernelLaunchFn kernel_fn;
7 };
```

All'interno troviamo:

- `buffer_size`: la dimensione dei due buffer;
- `size`: parametro usato dai thread per sapere quando fermarsi (spesso la dimensione intermedia di un risultato non coincide con quella del buffer iniziale);
- `src_buffer`: puntatore al buffer da processare;
- `dest_buffer`: puntatore al buffer del risultato;
- `kernel_fn`: puntatore alla funzione da simulare;

3.2.2 MyCudaKernelLaunch()

In questa funzione vengono creati i thread che andranno a eseguire la funzione puntata dal campo di un oggetto `MyCudaItem`. Per rispettare la struttura a griglia dell'esecuzione, i thread sono generati ad ogni iterazione di un singolo blocco, passandogli come funzione da eseguire proprio il parametro `kernel_fn`. Per gestire il passaggio dei parametri a ogni thread ho creato una struttura dati apposita,

```
1 struct ThreadArgs{
2     int threadIdx;
3     int blockIdx;
4     void* dest;
5     void* src;
```

```

6     int size;
7 };

```

inizializzata con `MyCudaSetThreadArgs()`. In questo modo ogni thread potrà accedere alla sua posizione nel buffer calcolando il proprio indice come

$$\text{Indice} = \text{IndiceThread} + \text{DimensioneBlocco} * \text{IndiceBlocco}$$

Con `MyCudaIterations()` vengono calcolati i passaggi totali per consumare l'input, ovvero il numero di blocchi totali.

```

1 void MyCudaKernelLaunch(MyCudaItem* cuda_item){
2     assert(cuda_item && "cuda_item not valid");
3     int passes = MyCudaIterations(cuda_item);
4     pthread_t threads[BLOCK_DIM];
5     ThreadArgs thread_arguments[BLOCK_DIM];
6     for(int bid = 0; bid < passes; bid++){
7         for(int tid = 0; tid < BLOCK_DIM; tid++){
8             MyCudaSetThreadArgs(&thread_arguments[tid], tid
9                                 , bid, cuda_item->dest_buffer, cuda_item->
10                                 src_buffer, cuda_item->size);
11             //kernel launch:
12             pthread_create(&threads[tid], NULL, cuda_item->
13                           kernel_fn, &thread_arguments[tid]);
14         }
15     }
16 }

```

Una volta entrati nella funzione, i parametri vengono estratti dalla `struct` e salvati localmente (vedere `MyCudaReductionKernel`).

3.3 Algoritmi

Vediamo due algoritmi di esempio implementati all'interno della libreria. Corrispondono alla versione parallela dei due visti nel capitolo 2.

3.3.1 MyCudaReduction

Restituisce la somma di tutte le componenti di un vettore in input.

Esempio di esecuzione:
src = 1 4 2 6 3 9 0 5
passo 1: dest = 5 8 12 5

```
passo 2:  dest = 13 17
passo 3:  dest = 30
```

MyCudaReduction() Questa funzione serve da "guscio" per l'esecuzione effettiva: crea i parametri, gestisce le iterazioni e invoca la (finta) chiamata al kernel. Dopo un primo passaggio il buffer di destinazione sarà grande metà dell'input e conterrà le somme a coppie degli elementi in **src**: a questo punto viene invocata **MyCudaMemSwap()** che inverte **dest** e **src** per il passaggio successivo, e così si itera finché la dimensione del risultato è pari a 1, ovvero la somma totale degli elementi iniziali.

```
1 void MyCudaReduction(MyCudaItem* cuda_item){
2     assert(cuda_item && "cuda_item_not_valid");
3
4     cuda_item->size = cuda_item->buffer_size;
5     cuda_item->size = (cuda_item->size+1)/2;
6     cuda_item->kernel_fn = MyCudaReductionKernel;
7
8     while(1){
9         //kernel launch
10        MyCudaKernelLaunch(cuda_item);
11        if(cuda_item->size>1){
12            cuda_item->size = (cuda_item->size+1)/2;
13            MyCudaMemSwap(cuda_item);
14        }else{
15            break;
16        }
17    }
18 }
```

MyCudaReductionKernel() L'esecuzione effettiva su GPU (o meglio la sua simulazione). Una volta estratti i parametri e verificato il giusto posizionamento entro i limiti del buffer, si sommano due celle contigue di **src** e viene scritto il risultato della corrispondente posizione in **dest**.

Nota: ogni thread ha due celle dedicate di **src** e una di **dest** e nessun altro interagisce con queste, preservando la correttezza del codice.

```
1 void* MyCudaReductionKernel(void* args){
2     ThreadArgs* info = (ThreadArgs*) args;
3     //estrazione parametri
4     int idx = info->threadIdx+info->blockIdx*BLOCK_DIM;
5     int size = info->size;
6     int* src = info->src;
```

```
7   int* dest = info->dest;
8
9   if(idx >= size){
10      pthread_exit(NULL);
11   }
12   dest[idx] = src[2*idx]+src[2*idx+1];
13   pthread_exit(NULL);
14 }
```

3.3.2 MyCudaPrefixSum

Preso il vettore in input, restituisce un nuovo vettore in cui in ogni posizione è scritta la somma dei valori dall'inizio fino a quella posizione del vettore in input. Questo algoritmo si divide in due passaggi:

- DownPass: scriviamo in **dest** le somme a coppie dell'input e il risultato viene poi sovrascritto nella rispettiva posizione pari della coppia in **src**.
- UpPass: sommiamo ogni valore di **src** nelle due celle rispettive di **dest**, saltando le prime 2 posizioni.

DownPass viene eseguito finchè la dimensione di **dest** è pari a 1. A quel punto invertiamo i due buffer e risaliamo invocando **UpPass** a ogni passaggio, finchè torniamo alla dimensione iniziale dell'input.

Nota: quando mi riferisco alla due celle rispettive di un buffer è perchè durante i passaggi troviamo sempre due buffer con le dimensioni uno metà dell'altro e viceversa, quindi possiamo mappare ogni cella del buffer piccolo a due celle del grande.

```
Esempio di esecuzione:
src = 1 2 3 4 2 3 1 4
passo 1a:
src = 1 2 3 4 2 3 1 4
dest = 3 7 5 5
passo 1b:
src = 1 3 3 7 2 5 1 5
dest = 3 7 5 5
passo 2a:
src = 3 7 5 5
dest = 10 10
passo 2b:
src = 3 10 5 10
dest = 10 10
passo 3a:
src = 10 10
dest = 20
```

```

passo 3b:
src = 10 20
dest = 20
passo 4:
src = 20
dest = 10 20
passo 5a:
src = 10 20
dest 3 10 5 10
passo 5b:
src = 10 20
dest 3 10 15 20
passo 6a:
src= 3 10 15 20
dest = 1 3 3 7 2 5 1 5
passo 6b:
src= 3 10 15 20
dest = 1 3 6 10 12 15 16 20

```

MyCudaPrefixSum() Anche stavolta ci serve una funzione di gestione che dall'esterno si occupa di parametri, esecuzioni e chiamate al kernel. Nella funzione gestisco le due chiamate a DownPass e UpPass nei momenti diversi semplicemente impostando il campo `kernel_fn` di `MyCudaItem` prima delle chiamate, sarà compito di **MyCudaKernelLaunch()** invocare la funzione puntata da quel campo.

Nota: Per evitare spreco di spazio, assumo negli algoritmi di avere dei buffer con dimensione potenza di due (verifica effettuata con funzione ausiliaria `MyCudaIsPowerTwo()`). In questo modo posso dividere il `dest` con dei sottobuffer contigui la cui dimensione si dimezza a ogni passaggio: la dimensione totale alla fine sarà perfettamente contenuta nel buffer. Con questo meccanismo posso evitare di allocare ulteriore memoria per i passaggi intermedi.

```

1 void MyCudaPrefixSum(MyCudaItem* cuda_item){
2     assert(cuda_item && "cuda_item_not_valid");
3     assert(MyCudaIsPowerTwo(cuda_item->buffer_size) && "Size
      must be a power of 2");
4
5     cuda_item->size = cuda_item->buffer_size;
6     cuda_item->size = (cuda_item->size+1)/2;
7     cuda_item->kernel_fn = MyCudaPrefixDownPassKernel;
8
9     int* orig_dests = cuda_item->dest_buffer;
10    int* orig_src = cuda_item->src_buffer;
11
12    while(cuda_item->size>0){
13        MyCudaKernelLaunch(cuda_item);

```

```

14     if(cuda_item->size == 1){
15         break;
16     }
17     cuda_item->src_buffer = cuda_item->dest_buffer;
18     cuda_item->dest_buffer += cuda_item->size;
19     cuda_item->size = (cuda_item->size+1)/2;
20 }
21 cuda_item->size = 2;
22
23 cuda_item->dest_buffer = cuda_item->src_buffer;
24
25 cuda_item->kernel_fn = MyCudaPrefixUpPassKernel;
26
27 while(cuda_item->size*2 < cuda_item->buffer_size){
28     cuda_item->src_buffer = cuda_item->dest_buffer;
29     cuda_item->dest_buffer -= cuda_item->size*2;
30     MyCudaKernelLaunch(cuda_item);
31     cuda_item->size *= 2;
32 }
33
34 cuda_item->dest_buffer = orig_src;
35 cuda_item->src_buffer = orig_dets;
36
37 //ultima chiamata per quando siamo tornati su 2 buffer
38 //diversi
39 MyCudaKernelLaunch(cuda_item);
40 }

```

MyCudaPrefixDownPassKernel() Questa è la funzione kernel di **DownPass**, invocata finché la dimensione del buffer di destinazione è 1.

```

1 void* MyCudaPrefixDownPassKernel(void* args){
2     ThreadArgs* info = (ThreadArgs*) args;
3
4     int idx = info->threadIdx+info->blockIdx*BLOCK_DIM;
5     int size = info->size;
6     int* src = info->src;
7     int* dest = info->dest;
8
9     if(idx >= size){
10         pthread_exit(NULL);
11     }
12
13     dest[idx] = src[2*idx]+src[2*idx +1];
14     src[2*idx +1] = dest[idx];
15 }

```

```
16 }

```

MyCudaPrefixUpPassKernel() Raggiunta la dimensione 1 possiamo risalire nella somma dei valori con **UpPass** fino a tornare alla dimensione iniziale. A quel punto il buffer di destinazione conterrà il risultato.

```

1 void* MyCudaPrefixUpPassKernel(void* args){
2     ThreadArgs* info = (ThreadArgs*) args;
3
4     int idx = info->threadIdx+info->blockIdx*BLOCK_DIM;
5     int size = info->size;
6     int* src = info->src;
7     int* dest = info->dest;
8
9     if(idx >= size){
10         pthread_exit(NULL);
11     }
12
13     if(2*(idx+1) < size*2){
14         dest[2*(idx+1)] += src[idx];
15     }
16     if(2*(idx+1)+1 < size*2){
17         dest[2*(idx+1)+1] += src[idx];
18     }
19 }

```

3.4 Funzioni Ausiliare

3.4.1 MyCudaMalloc()

Alloca i due buffer `src` e `dest` di `MyCudaItem`, aggiornando anche il campo della dimensione.

```

1 void MyCudaMalloc(MyCudaItem* cuda_item, int n){
2     assert(cuda_item && "CudaItem_not_valid");
3     int* res;
4     res = malloc(n*sizeof(int));
5     assert(res && "Memory_allocation_error_for_dest_MyCuda");
6     cuda_item->dest_buffer = res;
7     res = malloc(n*sizeof(int));
8     assert(res && "Memory_allocation_error_for_src_MyCuda");
9     cuda_item->src_buffer = res;

```

```
10     cuda_item->buffer_size = n;  
11 }
```

3.4.2 MyCudaMemSwap()

In molti algoritmi, prima di arrivare al risultato, ci sono diversi passaggi intermedi in cui si salvano risultati da usare successivamente. Per evitare di consumare molto spazio usiamo i due array di sorgente e destinazione in modo alternato. Questa funzione non fa altro che invertire, all'interno della struttura `MyCudaItem`, i puntatori alle due zone di memoria, azzerando i valori del nuovo buffer di destinazione. Ad esempio nel caso di `ReductionSum` si parte da `src` con dimensione `n`, si calcolano le somme a coppie e si salvano gli $n/2$ risultati in `dest`, al passaggio successivo il nuovo buffer sorgente è quello in cui abbiamo appena salvato le somme, quindi invertiamo i due array e ripetiamo finchè dopo $\log(n)$ passaggi troviamo un solo elemento che rappresenta il risultato.

```
1  
2 void MyCudaMemSwap(MyCudaItem* cuda_item){  
3     assert(cuda_item && "CudaItem not valid");  
4     int* aux = cuda_item->src_buffer;  
5     cuda_item->src_buffer = cuda_item->dest_buffer;  
6     cuda_item->dest_buffer = aux;  
7     for(int i = 0; i<cuda_item->buffer_size;i++){  
8         cuda_item->dest_buffer[i] = 0;  
9     }  
10 }
```

3.4.3 MyCudaMemFree()

Libera la memoria di un oggetto `MyCudaItem`.

```
1 void MyCudaMemFree(MyCudaItem* cuda_item){  
2     assert(cuda_item && "CudaItem not valid");  
3     free(cuda_item->dest_buffer);  
4     free(cuda_item->src_buffer);  
5     free(cuda_item);  
6 }
```

3.4.4 MyCudaBuffersPrint()

Stampa i due buffer di un oggetto MyCudaItem.

```

1 void MyCudaBuffersPrint(MyCudaItem* cuda_item){
2
3     assert(cuda_item && "CudaItem_not_valid");
4
5     int size = cuda_item->buffer_size;
6     int* src = cuda_item->src_buffer;
7     int* dest = cuda_item->dest_buffer;
8     printf("-----\n");
9     printf("SRC:\n");
10    for(int i = 0; i<size;i++){
11        printf("src [%d]=%d,\n",i,src[i]);
12    }
13    printf("DEST:\n");
14    for(int i = 0; i<size;i++){
15        printf("dest [%d]=%d,\n",i,dest[i]);
16    }
17    printf("-----\n");
18 }
```

3.4.5 MyCudaIterations()

Calcola le iterazioni necessarie per processare un buffer, ovvero quanti blocchi sono necessari per coprirne la dimensione totale.

```

1 int MyCudaIterations(MyCudaItem* item){
2     assert(item && "CudaItem_not_valid");
3     return (item->buffer_size+BLOCK_DIM-1)/BLOCK_DIM;
4 }
```

3.4.6 MyCudaSetThreadArgs()

Prepara gli argomenti da passare a ogni thread nella sua creazione.

```

1 void MyCudaSetThreadArgs(ThreadArgs* targs,int threadIdx, int
   blockIdx, int* dest, int* src, int size){
2     assert(targs && "ThreadArgs_not_valid");
3     targs->threadIdx = threadIdx;
```

```
4     targs->blockIdx = blockIdx;  
5     targs->dest = dest;  
6     targs->src = src;  
7     targs->size = size;  
8 }
```

3.4.7 MyCudaIsPowerTwo()

Verifica se un numero è una potenza di due.

```
1 int MyCudaIsPowerTwo(int n){  
2     int res = n;  
3     while(res>2){  
4         if(res%2){  
5             return 0;  
6         }  
7         res = res/2;  
8     }  
9     return 1;  
10 }
```

3.4.8 MyCudaPrefixSumTest()

Si occupa di verificare la correttezza di MyCudaPrefixSum comparando il risultato con quello di un algoritmo sequenziale. Gran parte della funzione consiste nella stampa dei risultati. Inizialmente genera dei valori casuali da inserire nel vettore di input, poi procede all'esecuzione nelle due modalità (sequenziale e parallela) e infine stampa i risultati affiancandoli in modo leggibile.

```
1 void MyCudaPrefixSumTest(MyCudaItem* cuda_item, int max_n){  
2  
3     int size = cuda_item->buffer_size;  
4     int test_buffer[size];  
5     int number;  
6  
7     printf("GENERATING_RANDOM_BUFFER(range_0-%d) OF SIZE [%d  
8         ]...\n", max_n, size);  
9     sleep(SLEEPING_TIME);  
10  
11     srand(time(NULL));  
12     for(int i = 0; i < size; i++){  
13         number = rand()%max_n;  
14     }
```

```
13     test_buffer[i] = number;
14     cuda_item->src_buffer[i] = number;
15 }
16 BufferPrint(test_buffer,size,"Test_Buffer");
17 BufferPrint(cuda_item->src_buffer,size,"Source_Buffer");
18
19 printf("SEQUENTIAL_PREFIX_SUM_ON_Test_Buffer...\n");
20 sleep(SLEEPING_TIME);
21
22 int sum = 0;
23 for(int i = 0;i<size;i++){
24     test_buffer[i] += sum;
25     sum = test_buffer[i];
26 }
27
28
29 printf("PARALLEL_PREFIX_SUM_ON_Source_Buffer...\n");
30 sleep(SLEEPING_TIME);
31
32 MyCudaPrefixSum(cuda_item);
33
34 printf("RESULTS:\n");
35 for(int i = 0;i<size;i++){
36     printf("Correct [%d]=%d, MyResult [%d]=%d\n",i,
37         test_buffer[i],i,cuda_item->dest_buffer[i]);
38 }
39 }
```

Uso e sviluppi

Come già detto non si tratta di una libreria che porta dei vantaggi prestazionali rispetto a un approccio tradizionale di programmare, lo scopo non è quello. L'obiettivo infatti è dare la possibilità a chiunque, anche chi non ha una GPU sul proprio computer, di poter scrivere e testare algoritmi che adottano la stessa meccanica che avrebbero se fossero scritti in CUDA. Una volta capite le funzionalità base infatti basta scrivere una funzione che rispetti le caratteristiche della computazione in parallelo. Mantenendo la coerenza dei protocolli di libreria (strutture dati ecc.) si può testare la funzione in pochi passaggi.

Al momento l'implementazione prevede unicamente l'utilizzo di vettori di interi, ma un possibile sviluppo della libreria potrebbe prevedere una riprogettazione della struttura `MyCudaItem` e di tutte le funzioni ausiliarie in modo da supportare la programmazione a oggetti, sfruttando meccanismi di ereditarietà e polimorfismo per poter aggiungere diversi tipi di dati. Dando uno sguardo a `MyCudaKernelLaunch()` si può notare subito la sua indipendenza dal tipo di dati trattati dalle funzioni, lasciando il compito al programmatore di occuparsi di questo aspetto e mantenendo una flessibilità sul tipo di funzioni da usare.

